

WAIT Engine

[2. Operationen \(DSL op\)](#)

[2.1. WAIT \(Pausen-Ausführung\)](#)

[2.1.1 Funktionsweise & Syntax-Toleranz](#)

[Unterstützte Zeitformate:](#)

[3. Referenz der WAIT-Parameter](#)

[Praxis-Tutorial: Umfassende WAIT-Engine Demonstration](#)

[Vollständiges YAML-Szenario](#)

[Log-Output der Engine](#)

[Troubleshooting & Best Practices](#)

[Troubleshooting-Checkliste](#)

[Performance-Hinweise](#)

Die WAIT-Engine ermöglicht gezielte Pausen innerhalb der Testausführung. Sie ist essenziell, um auf asynchrone Prozesse zu warten, Antwortzeiten von Systemen zu überbrücken oder Umgebungen Zeit zur Stabilisierung zu geben.

1. WAIT Engine - Kompakte System-Referenz für KI-Modelle

```
1 # AGATE WAIT-Engine Syntax Referenz
2 # Nutze dies für Testfall-Generierung und Analyse
3
4 - type: WAIT
5   value: "... "          # String/Zahl: "1000", "500ms", "2s", "1m" (De
6   condition: "... "      # Optional: '{B[var]}' == 'val' (einfache Anfü
7
```

2. Operationen (DSL op)

Im Gegensatz zu anderen Engines besitzt die WAIT-Engine keine verschiedenen Operationen (op). Sie führt standardmäßig eine Pausen-Operation basierend auf dem übergebenen Zeitwert aus.

2.1. WAIT (Pausen-Ausführung)

Blockiert die Testausführung für eine definierte Zeitspanne (mittels `Thread.sleep`) und setzt den Testlauf danach automatisch fort.

Parameter	Beschreibung	Erforderlich
-----------	--------------	--------------

value	Die Dauer der Pause. Unterstützt Zahlen (Millisekunden) und Zeiteinheiten (<code>ms</code> , <code>s</code> , <code>m</code>). Platzhalter werden vor der Ausführung aufgelöst.	Nein (Default: "0")
condition	Bedingte Ausführung. Wird der Schritt übersprungen (<code>SKIPPED</code>), wenn die Bedingung <code>false</code> ergibt.	Nein

2.1.1 Funktionsweise & Syntax-Toleranz

Die Engine bereitet den übergebenen String intelligent auf (`toLowerCase().trim().replaceAll("\\s+", "")`), bevor die Millisekunden berechnet werden. Das macht sie extrem tolerant gegenüber der Formatierung.

Unterstützte Zeitformate:

Format	Beschreibung	Berechnung im Hintergrund	Beispiel
ms	Millisekunden explicit	Direkt als Millisekunden	<code>500ms</code> , <code>500 MS</code>
s	Sekunden	Wert * \$1000\$	<code>2s</code> , <code>2 S</code>
m	Minuten	Wert * \$60 \cdot 1000\$	<code>1m</code> , <code>1 M</code>
Ohne Einheit	Reine Ganzzahl (Digit)	Interpretation als Millisekunden	<code>1000</code>
Leeres Feld / null	Fallback-Sicherheit	Wird automatisch als <code>0 ms</code> gewertet	<code>" "</code>

3. Referenz der WAIT-Parameter

Hier sind alle Parameter für den WAIT-Schritt im Überblick:

Schritt	Parameter	Beschreibung
WAIT	value	Der Zeitwert für die Pause (z. B. 1000 , 2s). Kann auch Variablen-Platzhalter enthalten.
	condition	<i>Optional:</i> Bedingung für die Ausführung. Der Schritt wird übersprungen (SKIPPED), wenn die Bedingung false ergibt.

Praxis-Tutorial: Umfassende WAIT-Engine Demonstration

Dieses Beispiel führt durch den Lebenszyklus eines Testfalls, der dynamische Timeouts verwendet und Bedingungen für Pausen prüft.

Vollständiges YAML-Szenario

Dieses Tutorial demonstriert die Flexibilität der Zeiteingaben und die Verwendung von Variablen im value -Feld.

1 | YAML

```
1 # WAIT Engine Demo Szenario
2 testCases:
3   - id: TC Wait Demonstration
4     description: Zeigt verschiedene Formate und Bedingungen der WAIT En
5     stage: "*"
6     priority: MEDIUM
7     variables:
8       dynamic_timeout: "1s"
9       skip_pause: "true"
10    steps:
11      # 1. Einfache Pause mit reiner Zahl (Millisekunden)
12      - type: WAIT
13        value: 1500
14
15      # 2. Pause mit explicit angegebener Sekundeneinheit und Leerzeich
16      - type: WAIT
17        value: "2 S"
18
19      # 3. Pause mit Minuteneinheit
20      - type: WAIT
21        value: "1m"
```

```

22
23     # 4. Dynamische Pause aus den Variablen (Buffer)
24     - type: WAIT
25       value: "{B[dynamic_timeout]}"
26
27     # 5. Bedingte Pause (wird ausgeführt, da Bedingung true)
28     - type: WAIT
29       condition: "'{B[dynamic_timeout]}' == '1s'"
30       value: "500ms"
31
32     # 6. Übersprungene Pause (Bedingung ergibt false)
33     - type: WAIT
34       condition: "'{B[skip_pause]}' == 'false'"
35       value: "5s"
36
37     # 7. Fallback-Sicherheit bei leeren Werten (Dauer: 0ms, bricht ni
38     - type: WAIT
39       value: ""
40

```

Log-Output der Engine

Wenn der Testfall im `isVerbose` -Modus läuft, loggt die Engine die berechnete Wartezeit transparent, bevor der Thread schläft:

```

1  >>> DURATION   : 1500 ms
2  >>> DURATION   : 2000 ms
3  >>> DURATION   : 60000 ms
4  >>> DURATION   : 1000 ms

```

Troubleshooting & Best Practices

Wenn deine Pausen nicht wie erwartet funktionieren, überprüfe die folgenden häufigen Ursachen:

Troubleshooting-Checkliste

- **Test bricht mit `RuntimeException` ab:**

- *Meldung:* `Invalid number format in WAIT step` oder `Unknown time format`.
- *Ursache:* Es wurde ein ungültiges Format übergeben (z. B. `10sec`, `19Years` oder ein ungültiger Variablenname, der nicht aufgelöst werden konnte). Das Framework nutzt ein striktes Fail-Fast-Prinzip, um unbemerkt fehlende Wartezeiten zu verhindern.

- **Condition wird immer übersprungen:**

- Achte auf die korrekte Syntax mit einfachen Anführungszeichen: `condition: "{B[my_var]}' == 'val'"`.

- **Thread Interruption im Log:**

- Wenn der Testlauf hart abgebrochen wird (z. B. durch den Benutzer oder CI/CD-Server), fängt die Engine die `InterruptedException` ab, setzt den Interrupt-Status zurück und wirft eine saubere `RuntimeException("Wait interrupted: ...")`.

Performance-Hinweise

- **Kein Ersatz für echte Synchronisation:** Nutze die WAIT-Engine sparsam. "Hard-Coded"
Pausen (z. B. pauschal 10s warten) verlangsamen die Testsuite unnötig. Es ist Best Practice, stattdessen über ASSERT-Schritte der anderen Engines (z. B. SQL oder CMD) gezielt auf das Eintreffen eines Zustands zu prüfen, sofern das Zielsystem dies erlaubt.